



FORMATION PROMETHEUS & GRAFANA

Guide stagiaire — Dernier jour, version pratique

Pratiquer : de la requête au tableau de bord, puis à l'alerte

Sommaire

Pratiquer : de la requête au tableau de bord, puis à l'alerte	2
Le programme du jour	2
Rattrapage — le lab prêt en une commande	2
Mission 0 — La chasse aux métriques	3
L'aide-mémoire PromQL : huit formes à connaître	3
Missions 1 à 8 — PromQL dans Grafana Explore	4
Rappels — Grafana	6
TP A — Le tableau de bord d'un serveur Linux	6
TP B — Le tableau de bord de la boutique	8
TP C — Alerter	11
War game en binôme	12
Ce qu'on n'a pas eu le temps de faire, et où le trouver	12
Lundi matin	13

Pratiquer : de la requête au tableau de bord, puis à l'alerte

Le programme du jour

Aujourd'hui, on ne code pas : l'application est livrée instrumentée. On passe la journée à **interroger**, **visualiser**, **casser** et **alerter**.

Heure	Séquence
9h00	Rattrapage : le lab dans l'état « application instrumentée »
9h20	Mission 0 — La chasse aux métriques
9h35	Missions 1 à 8 — PromQL dans Grafana Explore
10h30	<i>Pause</i>
11h00	Grafana en 15 minutes, puis TP A — Le tableau de bord d'un serveur Linux
12h20	Le dashboard de la communauté en 2 minutes
12h30	<i>Déjeuner</i>
13h30	TP B — Le tableau de bord de la boutique
14h30	<i>Pause</i>
15h00	TP C — Une alerte Prometheus, puis une alerte Grafana
15h35	War game en binôme
15h50	Bilan, lundi matin
16h00	Fin

Rattrapage — le lab prêt en une commande

Dans le terminal de votre Codespace (ou de votre machine), à la racine du dépôt :

```
git fetch origin
git checkout origin/formation-2026 -- rattrapage/
bash rattrapage/appliquer.sh
```

Le script sauvegarde vos fichiers dans `rattrapage/sauvegarde-<date>/`, active les briques 01 à 06, dépose l'application instrumentée et la configuration Prometheus complète, démarre la stack et affiche l'état des cibles. La première fois, comptez 2 à 4 minutes.

Résultat attendu : 10 / 10 cibles UP à la fin du script. Sinon, relancez-le une fois ; si c'est toujours rouge, levez la main.

Les onglets à ouvrir (onglet **PORTS** du Codespace, icône globe) :

Port	Service	Identifiants
3000	Grafana	admin / formation

Port	Service	Identifiants
9090	Prometheus	
9093	Alertmanager	
8080	Inbox (les notifications arrivent ici)	
5001	shop-api-1, page <code>/metrics</code>	

Mission 0 — La chasse aux métriques 10 min

Ouvrez `/metrics` sur le port 5001. C'est ce que l'application expose : du texte, une ligne par série. Retrouvez un exemple de chacun des quatre types et remplissez le tableau.

Type	Nom de la métrique	Ses labels	Ce qu'elle mesure
Counter			
Gauge			
Histogram			
Summary			

Questions :

- Combien de lignes `http_request_duration_seconds_bucket` pour **une** route ? Que veut dire `le="0.25"` ?
- Pourquoi `shop_orders_total` finit par `_total` et pas `shop_stock_units` ?
- `shop_app_info` vaut toujours 1. À quoi peut-elle bien servir ?

L'aide-mémoire PromQL : huit formes à connaître

Vous n'avez pas besoin de connaître PromQL par cœur : ces huit formes couvrent 90 % des tableaux de bord. Pour chaque mission, trouvez la bonne forme et adaptez-la.

#	Forme	À quoi ça sert
1	<code>metrique{label="valeur"}</code>	filtrer (<code>=</code> , <code>!=</code> , <code>=~</code> regex, <code>!~</code>)
2	<code>rate(compteur[5m])</code>	la vitesse d'un compteur, par seconde
3	<code>sum by (label) (...)</code>	additionner en gardant un label (<code>avg</code> , <code>max</code> , <code>count</code> aussi)
4	<code>increase(compteur[1h])</code>	combien en une heure (et non plus par seconde)
5	<code>sum(rate(...{status=~"5.."}[5m])) / sum(rate(...[5m]))</code>	un ratio , ici le taux d'erreur
6	<code>histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))</code>	le p95 d'une latence

#	Forme	À quoi ça sert
7	<code>topk(3, ...) / bottomk(3, ...)</code>	les 3 plus grands / plus petits
8	<code>expression > seuil / == 0</code>	ne garder que ce qui dépasse, c'est une alerte

Et deux règles d'or :

- un **compteur** ne se regarde jamais brut : toujours `rate` OU `increase` ;
- `rate` d'abord, `sum` ensuite (jamais l'inverse).

Missions 1 à 8 — PromQL dans Grafana Explore 55 min

Grafana → menu **Explore** → source **Prometheus** → mode **Code** (bouton en haut à droite de l'éditeur). Notez vos requêtes : plusieurs serviront telles quelles cet après-midi.

Mission 1 — Qui est vivant ?

1. Affichez toutes les cibles et leur état. Combien y en a-t-il ?
2. N'affichez que les cibles **en panne**. Que voyez-vous quand tout va bien ?
3. Comptez les cibles par `job`.

Mission 2 — Combien de requêtes ?

1. Affichez `http_requests_total` en mode Graph. Pourquoi la courbe ne fait-elle que monter ?
2. Transformez-la en requêtes **par seconde**.
3. Une seule courbe : le débit total de la boutique. Puis une courbe par `route`.

Mission 3 — Combien d'erreurs ?

1. Le débit des seules réponses 5xx.
2. Le **taux d'erreur** : la part des 5xx dans le total (entre 0 et 1).
3. Dans le terminal : `./lab.sh chaos errors on`. Relancez la requête. Qu'observez-vous au bout d'une minute ?
Puis `./lab.sh chaos errors off`.

Mission 4 — Est-ce que c'est lent ?

1. Le p95 de la latence de toute la boutique.

2. Le p95 **par route**. Laquelle est la plus lente ?
3. `./lab.sh chaos latency on`, observez, puis `off`.

Mission 5 — Et le business ?

1. Le chiffre d'affaires **par heure** (en euros).
2. Le nombre de commandes sur la dernière heure.
3. Les trois produits dont le stock est le plus bas.

Mission 6 — Le serveur

1. Le pourcentage de CPU utilisé (indice : 100 moins le temps passé en mode `idle`).
2. Le pourcentage de mémoire utilisée (`node_memory_MemAvailable_bytes` , `node_memory_MemTotal_bytes`).
3. `./lab.sh chaos cpu 120` : retrouvez le pic.

Mission 7 — Vu de l'extérieur

1. Quelles sondes Blackbox réussissent ? Laquelle est la plus lente (`probe_duration_seconds`) ?
2. Redis est-il vivant (`redis_up`) ? Combien de clients connectés ?

Mission 8 — Le batch de la nuit

Lancez `./lab.sh batch`. Puis : depuis combien de **secondes** la dernière sauvegarde a-t-elle réussi ? (indice : `time()` donne l'heure actuelle ; cherchez une métrique `backup_...`)

Pour les rapides : dans Explore, cliquez sur Split et comparez le débit de `shop-api-1` et de `shop-api-2` côte à côte.

Rappels — Grafana

Concepts. Data source → Panel (une requête + une visualisation + des options) → Dashboard (panels, variables, annotations, liens) → Folder (rangement et droits).

Grafana 13. Barre latérale d'édition (*Add* : Panel, Add row, Variable, Annotation query, Link), éditeur de panel avec les requêtes en bas et les options à droite.

Choisir la visualisation. Évolution → Time series. Valeur actuelle → Stat. Niveau borné → Gauge. Comparer quelques valeurs → Bar gauge. Liste → Table. Répartition → Pie chart. État dans le temps → State timeline. Distribution → Heatmap.

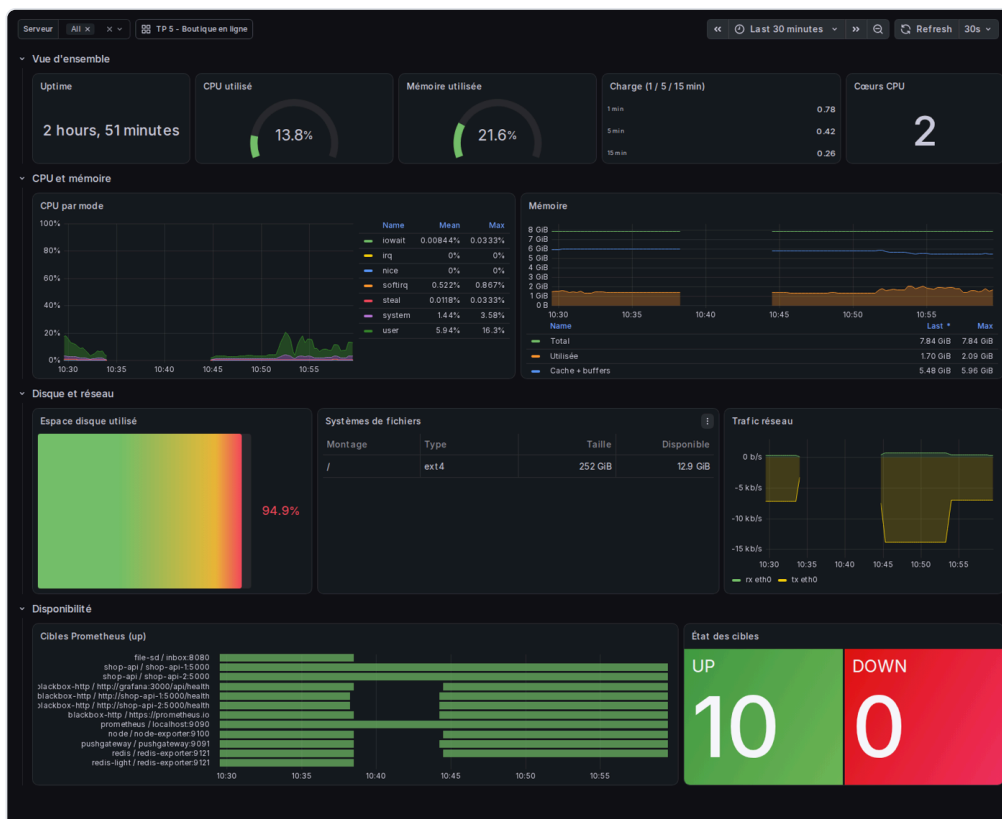
Les trois réglages qui changent tout. L'unité (*Standard options* → *Unit*), les seuils (*Thresholds*), la légende (`{{label}}`).

Variables. `$instance` dans les requêtes, avec `=~"$instance"` pour accepter plusieurs valeurs. `$_rate_interval` à la place de `[5m]` : Grafana choisit la bonne fenêtre selon le zoom.

TP A — Le tableau de bord d'un serveur Linux 80 min

Situation. L'équipe d'exploitation veut un écran par serveur : d'un coup d'œil, savoir s'il va bien ; en dessous, le détail. Une liste déroulante pour choisir le serveur.

Les requêtes sont fournies : votre travail, c'est Grafana. Chaque panel doit être **lisible par quelqu'un qui n'a pas suivi la formation** : un titre clair, la bonne unité, des seuils.



Le résultat attendu (le vôtre aura moins de panels : c'est normal)

Étape 0 — Le dashboard et sa variable 10 min

1. Dashboards → New → New dashboard.

2. **Add → Variable** : type *Query*, nom `instance`, label `Serveur`, requête `label_values(node_uname_info, instance)`. Cochez *Multi-value* et *Include All value* (valeur personnalisée pour All : `.*`).
3. Sauvegardez : TP A - Serveur Linux, dossier *Formation*.

Étape 1 — Row « Vue d'ensemble » 25 min

Add → Row, puis un panel à la fois. Pour chaque panel : collez la requête, choisissez la visualisation, puis réglez les options indiquées (champ de recherche en haut des options).

#	Panel	Visualisation	Requête	Réglages
1	Uptime	Stat	<code>time() - node_boot_time_seconds{instance=~"\$instance"}</code>	Unit <i>duration</i> (<i>dt</i> durations) ; requête en <i>Instant</i>
2	CPU utilisé	Gauge	<code>100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle", instance=~"\$instance"}[\$__rate_interval])))</code>	Unit <i>Percent</i> (0-100), Min 0, Max 100, Thresholds vert / orange 70 / rouge 90
3	Mémoire utilisée	Gauge	<code>100 * (1 - node_memory_MemAvailable_bytes{instance=~"\$instance"} / node_memory_MemTotal_bytes{instance=~"\$instance"})</code>	comme le CPU
4	Charge 1 / 5 / 15 min	Stat	trois requêtes : <code>node_load1{instance=~"\$instance"}</code> , puis <code>node_load5</code> , <code>node_load15</code>	Legend 1 min, 5 min, 15 min ; 2 décimales
5	Cœurs CPU	Stat	<code>count(node_cpu_seconds_total{mode="idle", instance=~"\$instance"})</code>	couleur fixe bleue

Vérification : le CPU est-il vert ? Lancez `./lab.sh chaos cpu 120` et regardez la jauge changer de couleur (rafraîchissement automatique : menu en haut à droite, 10 s).

Étape 2 — Row « CPU et mémoire » 15 min

#	Panel	Visualisation	Requête	Réglages
6	CPU par mode	Time series	<code>sum by (mode) (rate(node_cpu_seconds_total{mode!="idle", instance=~"\$instance"}[\$__rate_interval])) / scalar(count(node_cpu_seconds_total{mode="idle", instance=~"\$instance"}))</code>	Legend <code>{{mode}}</code> ; <i>Stack series</i> : <i>Normal</i> ; <i>Fill opacity</i> 40 ; Unit <i>Percent</i> (0.0-1.0) ; légende en <i>Table</i> à droite avec <i>Mean</i> et <i>Max</i>
7	Mémoire	Time series	Total : <code>node_memory_MemTotal_bytes{instance=~"\$instance"}</code> ; Utilisée : Total - <code>node_memory_MemAvailable_bytes</code>	Unit <i>bytes</i> (IEC) ; <i>Add field override</i> sur « Utilisée » : couleur orange

Question : pourquoi `scalar()` dans le panel 6 ?

Étape 3 — Row « Disque et disponibilité » 20 min

#	Panel	Visualisation	Requête	Réglages
8	Espace disque utilisé	Bar gauge	<code>100 * (1 - node_filesystem_avail_bytes{fstype!~"tmpfs overlay squashfs", instance=~"\$instance"} / node_filesystem_size_bytes{fstype!~"tmpfs overlay squashfs", instance=~"\$instance"})</code>	Legend <code>{{mountpoint}}</code> ; Instant ; horizontal, mode <i>Gradient</i> ; Percent ; seuils 75 / 90
9	Cibles Prometheus	State timeline	<code>up</code>	Legend <code>{{job}}</code> / <code>{{instance}}</code> ; <i>Value mappings</i> : 1 → UP en vert, 0 → DOWN en rouge
10	Cibles en panne	Stat	<code>count(up == 0) or vector(0)</code>	<i>Color mode</i> : <i>Background</i> ; seuils vert, puis rouge à 1

Testez votre dashboard : `docker stop shop-api-2`, attendez 30 s. Qu'est-ce qui passe au rouge ? Puis `docker start shop-api-2`.

Question : que se passe-t-il sur le panel 10 sans `or vector(0)` quand tout va bien ?

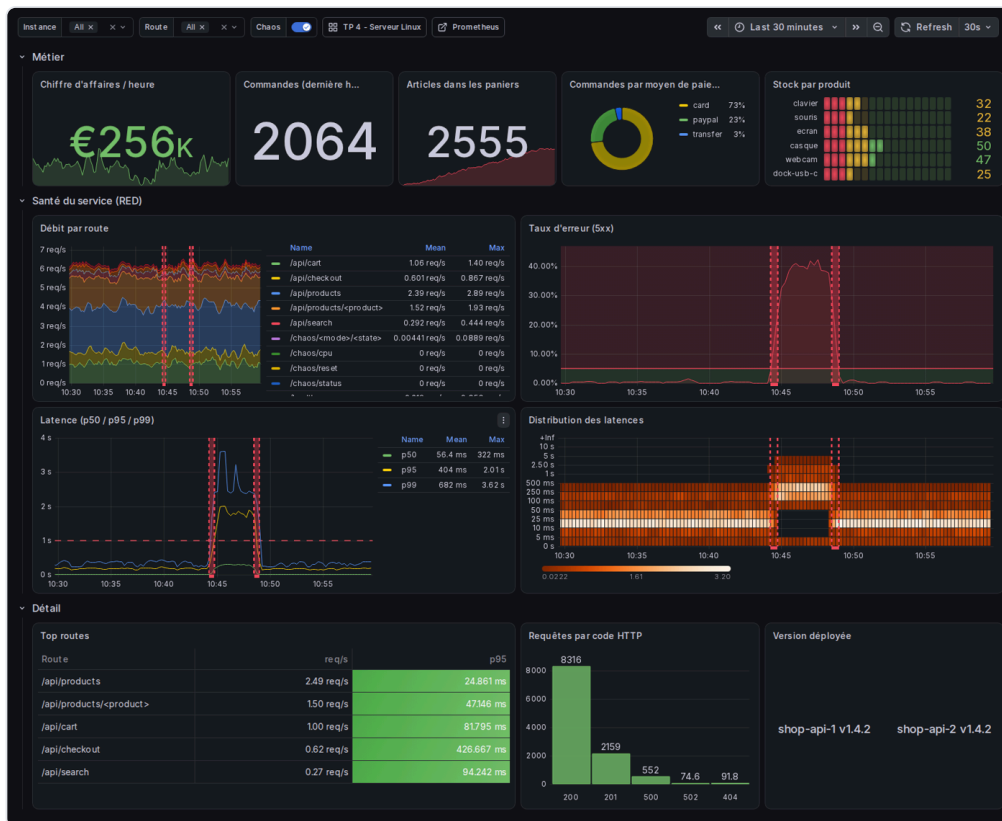
Pour les rapides : un panel Trafic réseau (Time series), réception en positif et émission en négatif, avec `rate(node_network_receive_bytes_total{device!~"lo|veth.*|br.*|docker.*"}[$__rate_interval]) * 8` et le même en `transmit` multiplié par `-8`, unité bits/sec.

Le dashboard de la communauté 5 min

Dashboards → **New** → **Import**, ID `1860`, *Load*, source Prometheus, *Import*. C'est « Node Exporter Full », le dashboard le plus téléchargé de grafana.com. Comparez avec le vôtre : lequel monteriez-vous à un astreinte à 3 h du matin ?

TP B — Le tableau de bord de la boutique 60 min

Situation. Le directeur commercial veut le chiffre d'affaires, les commandes, les stocks. L'équipe technique veut, sur le même écran, débit, erreurs et latence. Et tout le monde veut voir sur les graphiques « quand est-ce qu'on a cassé quelque chose ».



Le résultat attendu

Étape 0 — Le dashboard et sa variable 5 min

Nouveau dashboard `TP B - Boutique en ligne`, dossier *Formation*. Variable *Query* `instance`, multi-valeur avec `All` : `label_values(http_requests_total{job="shop-api"}, instance)`.

Toutes les requêtes ci-dessous filtrent sur `job="shop-api", instance=~"$instance"` : c'est ce qui relie chaque panel à la liste déroulante.

Étape 1 — Row « Métier » 20 min

#	Panel	Visualisation	Requête	Réglages
1	Chiffre d'affaires / heure	Stat	<code>sum(rate(shop_revenue_euros_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval])) * 3600</code>	Unit <i>Euro</i> (€) ; 0 décimale ; <i>Graph mode</i> : <i>Area</i> ; couleur verte
2	Commandes (1 h)	Stat	<code>sum(increase(shop_orders_total{job="shop-api", instance=~"\$instance"}[1h]))</code>	Instant ; 0 décimale
3	Moyens de paiement	Pie chart	<code>sum by (payment_method) (increase(shop_orders_total{job="shop-api", instance=~"\$instance"}[\$__range]))</code>	Legend <code>{{payment_method}}</code> ; Instant ; <i>Pie chart type</i> : <i>Donut</i> ; légende à droite avec <i>Percent</i>
4	Stock par produit	Bar gauge	<code>avg by (product) (shop_stock_units{job="shop-api", instance=~"\$instance"})</code>	Legend <code>{{product}}</code> ; Instant ; mode <i>LCD</i> ; Max 120 ; seuils rouge, orange à 20, vert à 40

Question : pourquoi `rate` × 3600 plutôt que la valeur brute de `shop_revenue_euros_total` ?

Étape 2 — Row « Santé du service » 25 min

Les trois signaux qu'on regarde en premier sur tout service : **Rate** (débit), **Errors**, **Duration** (latence).

#	Panel	Visualisation	Requête	Réglages
5	Débit par route	Time series	<code>sum by (route) (rate(http_requests_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval]))</code>	Legend <code>{{route}}</code> ; empilé ; Unit <code>requests/sec (rps)</code>
6	Taux d'erreur	Time series	<code>sum(rate(http_requests_total{job="shop-api", instance=~"\$instance", status=~"5.."}[\$__rate_interval])) / sum(rate(http_requests_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval]))</code>	Unit <code>Percent (0.0-1.0)</code> ; seuil rouge à 0.05 ; <i>Show thresholds : As lines and filled regions</i> ; couleur rouge
7	Latence p50 / p95 / p99	Time series	<code>histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api", instance=~"\$instance"}[\$__rate_interval])))</code> , puis la même avec 0.50 et 0.99	Legend <code>p95</code> , <code>p50</code> , <code>p99</code> ; Unit <code>seconds (s)</code> ; seuil pointillé à 1
8	Distribution des latences	Heatmap	<code>sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api", instance=~"\$instance"}[\$__rate_interval]))</code>	<i>Format : Heatmap</i> (sous la requête) ; <i>Calculate from data : No</i> ; palette <i>Oranges</i>

Lancez `./lab.sh chaos latency on` pendant trois minutes : regardez la heatmap et le p99, puis `off` .

Étape 3 — Voir les incidents sur les courbes 10 min

- Add → Annotation query** : nom `Chaos` , source Prometheus, requête `changes(shop_chaos_mode{instance=~"$instance"}[1m]) > 0` , titre `Chaos {{mode}}` , couleur rouge.
- Add → Link** : type *Dashboard*, vers `TP A - Serveur Linux` , cochez *Keep time range*.
- `./lab.sh chaos errors on` , deux minutes, puis `off` . Des traits rouges verticaux apparaissent sur toutes les courbes, au moment exact de la panne.

Pour les rapides : *Export → Export as code*, copiez le JSON dans `grafana/dashboards/tp-b-boutique.json` , attendez 10 s et rechargez la liste des dashboards. Le même dashboard existe maintenant « en code », versionnable dans Git.

TP C — Alerter 35 min

Partie 1 — Une alerte Prometheus, de bout en bout 10 min

Une règle existe déjà dans `prometheus/rules/alerts.yml` : `TargetDown`, qui se déclenche quand `up == 0` pendant une minute. Suivez-la de bout en bout :

1. Ouvrez côte à côte : Prometheus **Alerts**, l'Alertmanager (9093), l'Inbox (8080).
2. `docker stop shop-api-1`.
3. Notez l'heure de chaque étape : l'alerte passe *Pending*, puis *Firing* dans Prometheus, puis apparaît dans l'Alertmanager, puis arrive dans l'Inbox.
4. `docker start shop-api-1`. La notification « resolved » arrive dans l'Inbox, mais pas tout de suite : jusqu'à 5 minutes. Cherchez `group_interval` dans `alertmanager/alertmanager.yml`.

Pending à ... · Firing à ... · Inbox à ... · Résolue à ...

Question : pourquoi un délai d'environ 1 min 30 entre la panne et la notification ? Est-ce un défaut ?

Partie 2 — Une alerte Grafana sur le taux d'erreur 25 min

1. **Alerting** → **Notification configuration** → **Contact points** → **New contact point** : nom `inbox-grafana`, intégration *Webhook*, URL `http://inbox:8080/webhook/grafana`. **Test**, vérifiez l'Inbox, sauvegardez.
2. **Alerting** → **Alert rules** → **New alert rule** : - nom `Boutique - taux d'erreur élevé` ; - requête A, mode *Code*, type **Instant** : `100 * sum by (instance) (rate(http_requests_total{job="shop-api", status=~"5.."}[5m])) / sum by (instance) (rate(http_requests_total{job="shop-api"}[5m]))` ;
 - condition : *IS ABOVE* `5`, puis *Preview* ;
 - dossier *Formation*, labels `severity=critical`, `team=boutique` ;
 - groupe d'évaluation `boutique`, intervalle 1 min, *Pending period* 1 min ;
 - notifications : choisissez directement le contact point `inbox-grafana` ;
 - *Summary* : `{{ $labels.instance }}` : trop d'erreurs ;
 - *Link dashboard and panel* : votre panel « Taux d'erreur » du TP B. 3. Sauvegardez. `./lab.sh chaos errors on`. Suivez l'état de la règle (*Normal* → *Pending* → *Firing*), puis l'Inbox. Retournez sur le dashboard TP B : l'alerte apparaît sur le panel lié.
4. `./lab.sh chaos errors off`.

Question : Prometheus + Alertmanager, ou Grafana ? Dans quel cas choisiriez-vous l'un ou l'autre ?

War game en binôme 15 min

Par deux, sur **un seul écran**. Avant de jouer, décochez l'annotation « Chaos » en haut du dashboard TP B (sinon elle donne la réponse). A se retourne. B tire une carte au hasard, lance la commande dans le terminal, ferme le terminal. A a **cinq minutes** pour trouver ce qui ne va pas, avec **uniquement** Grafana (vos dashboards, Explore) et les pages Prometheus / Alertmanager. Puis on remet en ordre, et on inverse.

Carte	Commande (secrète)
1	<code>./lab.sh chaos latency on</code>
2	<code>./lab.sh chaos errors on</code>
3	<code>curl -s -X POST localhost:5002/chaos/errors/on</code>
4	<code>./lab.sh traffic 40</code>
5	<code>./lab.sh chaos cpu 180</code>
6	<code>docker stop shop-api-1</code>
7	<code>docker stop redis</code>
8	<code>./lab.sh chaos leak on</code>

Remise en ordre après chaque manche :

```
./lab.sh chaos reset && ./lab.sh traffic 6 && docker start shop-api-1 redis
```

(La carte 5 s'arrête toute seule au bout de 3 minutes.)

Grille du diagnostiqueur :

	Manche 1	Manche 2
Premier symptôme vu (quel panel ?)		
Quoi ? (débit, erreurs, latence, saturation, panne)		
Où ? (quelle instance, quel service)		
Depuis quand ?		
Temps pour trouver		

Les cartes 7 et 8 sont les plus dures : aucun de vos dashboards ne les montre directement. Pensez à Explore, `redis_up` et `process_resident_memory_bytes`.

Ce qu'on n'a pas eu le temps de faire, et où le trouver

Tout est dans le dépôt, avec les guides : faisables chez vous, dans un Codespace, à votre rythme.

Sujet	Où
Écrire l'instrumentation soi-même	Guide jour 1, TP 2 (les cinq TODO de <code>apps/shop-api/app.py</code>)
Recording rules et tests unitaires	Guide jour 2, TP 3 (les règles sont déjà dans <code>prometheus/rules/recording.yml</code>)
Droits, équipes, service accounts	Guide jour 2, exercices 2.30 à 2.33

Sujet	Où
Routage Alertmanager, inhibition, silences	Guide jour 3, TP 6
Notifications Teams	Guide jour 3, TP 7
Alerting Grafana en code	Guide jour 3, TP 8 partie 4
Sauvegarde, restauration, sécurité	Guide jour 3, TP 9
Thanos : historique long, vue globale	Guide jour 3, TP 10

Lundi matin

Choisissez **un** service de votre périmètre. Un seul. Vérifiez qu'il expose des métriques (ou qu'un exporter existe), faites-le scraper, et construisez-lui une row RED : débit, erreurs, latence. Puis **une** alerte sur un symptôme que vos utilisateurs sentiraient. C'est tout. Le reste viendra.

Mon service : ... · Son exporter : ... · Mon alerte : ...